# Introduction to Software Architecture



Davide Falessi

# Agenda

- What is a software architecture
- How to find it
- How to document it
- Product lines
- Architectural styles

# What is a software architecture?

# What is a software architecture

- A commonly accepted definition of software architecture comes from the IEEE 1471 standard.

**"The fundamental organization of a system, embodied in its components, their relationships to each other and to the environment, and the principles governing its design and evolution."**

# SOFTWARE ARCHITECTURE: AIMS

"The main intent of software architecture is to provide **intellectual control** over a sophisticated system of enormous complexity."
(Kruchten)

# SOFTWARE ARCHITECTURE: NEEDS









# SOFTWARE ARCHITECTURE: ALWAYS

- Every system has its own architecture.

- **Intended** (planned, designed)
  - Explicit
  - Documented

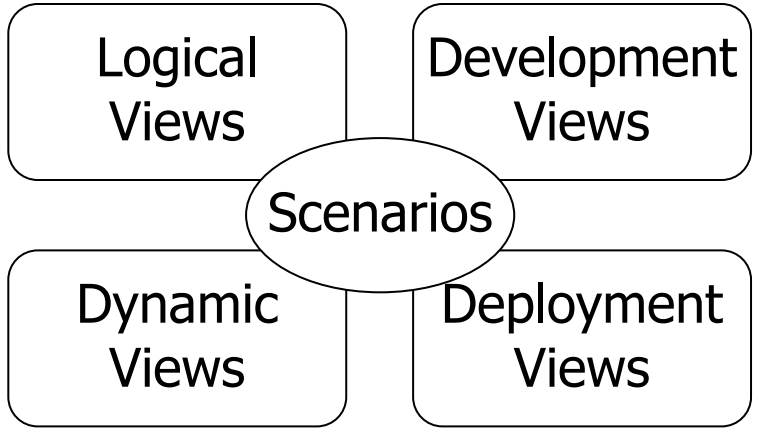
- **Unintended** (spontaneous, accidental)
  - Implicit
  - To be reconstructed
  - To be documented.

# SOFTWARE ARCHITECTURE: CORE PRINCIPLE

- The separation of concerns and views (4+1) breaks down the complexity of the system allowing for better management. (Kruchten)

# SA: 4+1

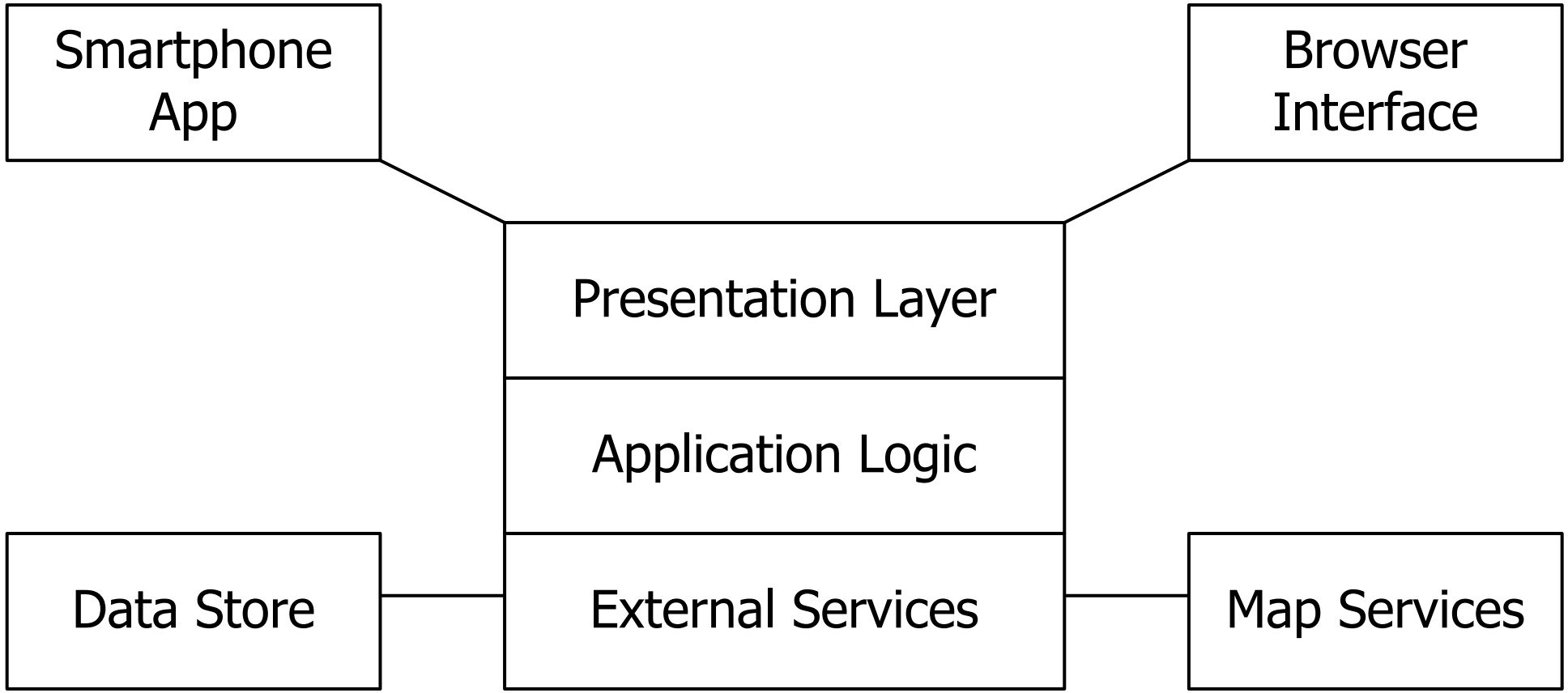
- Logical Views focus on support for user functionality and requirements
- Development Views focus on modules in the source code
- Dynamic Views focus on run-time object/process, data flows
- Deployment Views focus on configuration, mapping of software onto servers, geographic distribution




Kruchten [1995]

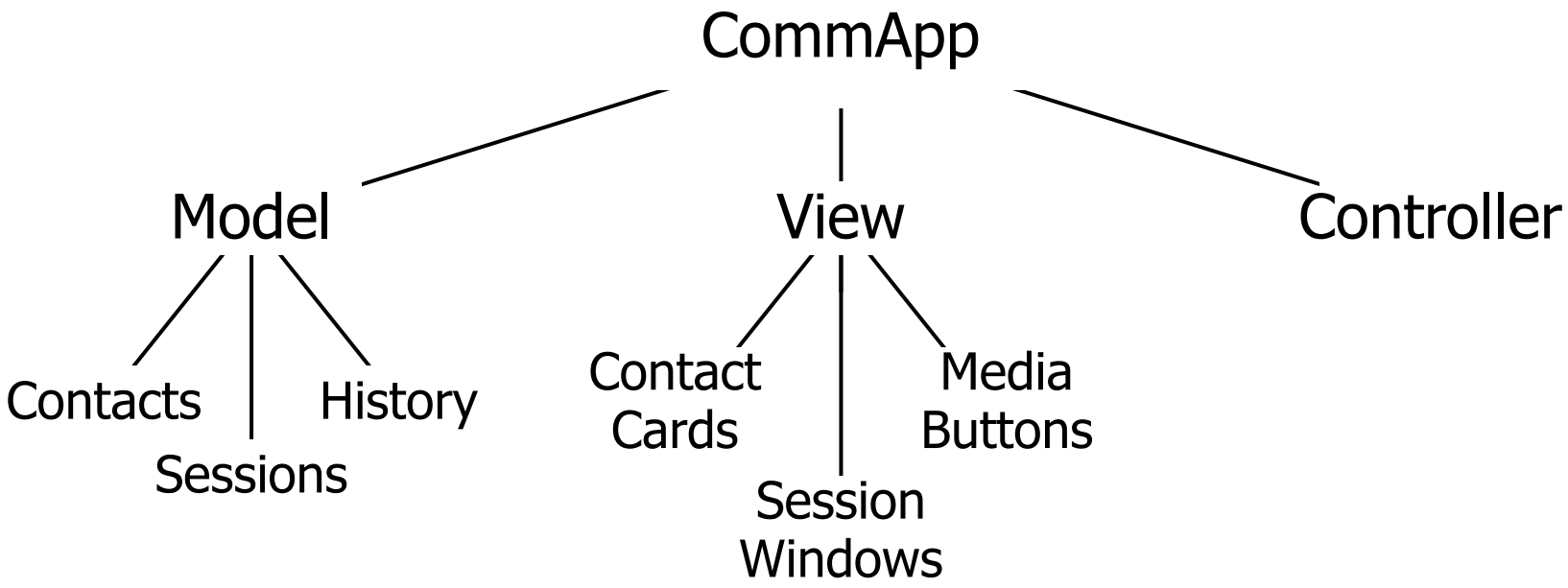
# A Logical View of a System

The system has two user interfaces: for smartphone and browser



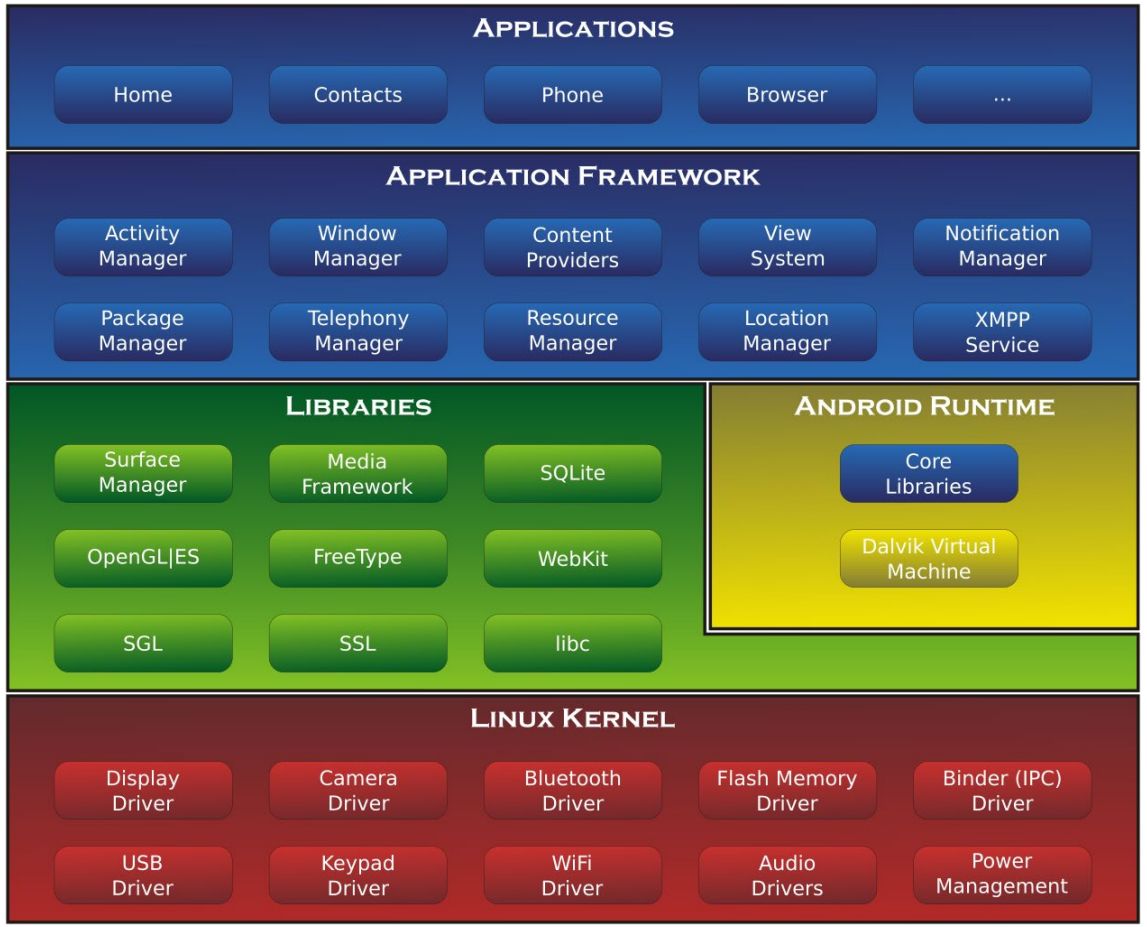
# A Development View: A Module Hierarchy

This view shows module-submodule relationships



# A Development View of the Android Stack

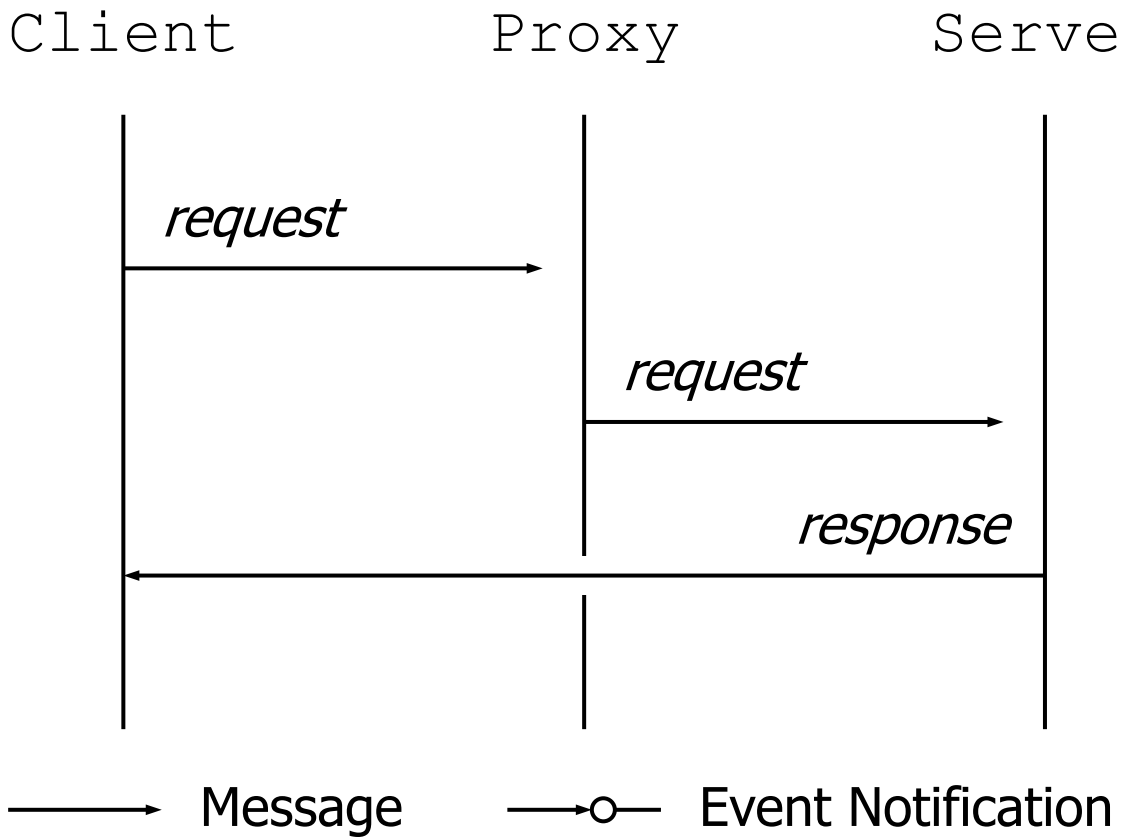
## Google's diagram of the Android Software Stack




By Smieh - Anatomy Physiology of an Android, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=20067152

# A Dynamic View of Interactions

This view focuses on interactions rather than on the components



# How to find a SA?

# Software architecture design

- Software architecture design methods aim to derive the software architecture from the functional and quality requirements.

- There are three main factors that drive  software architecture design: reuse, method, and intuition.

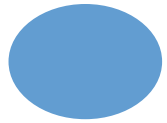
# Software architecture design

- **Reuse**: Architecting is a difficult task that reuse can help simplify.

  The fact that a system was successful and it used a certain component, rule, or responsibility argues for the reuse of such issues.

The source of the reuse can be an **earlier version** of the system, another system sharing key characteristics, architectural patterns, or organization-wide experiences.

*Domain-specific Software Architecture* and *Product Line Architecture* are examples of two successful marriages between software architecture and reuse.

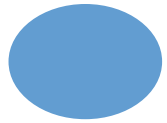


# Software architecture design

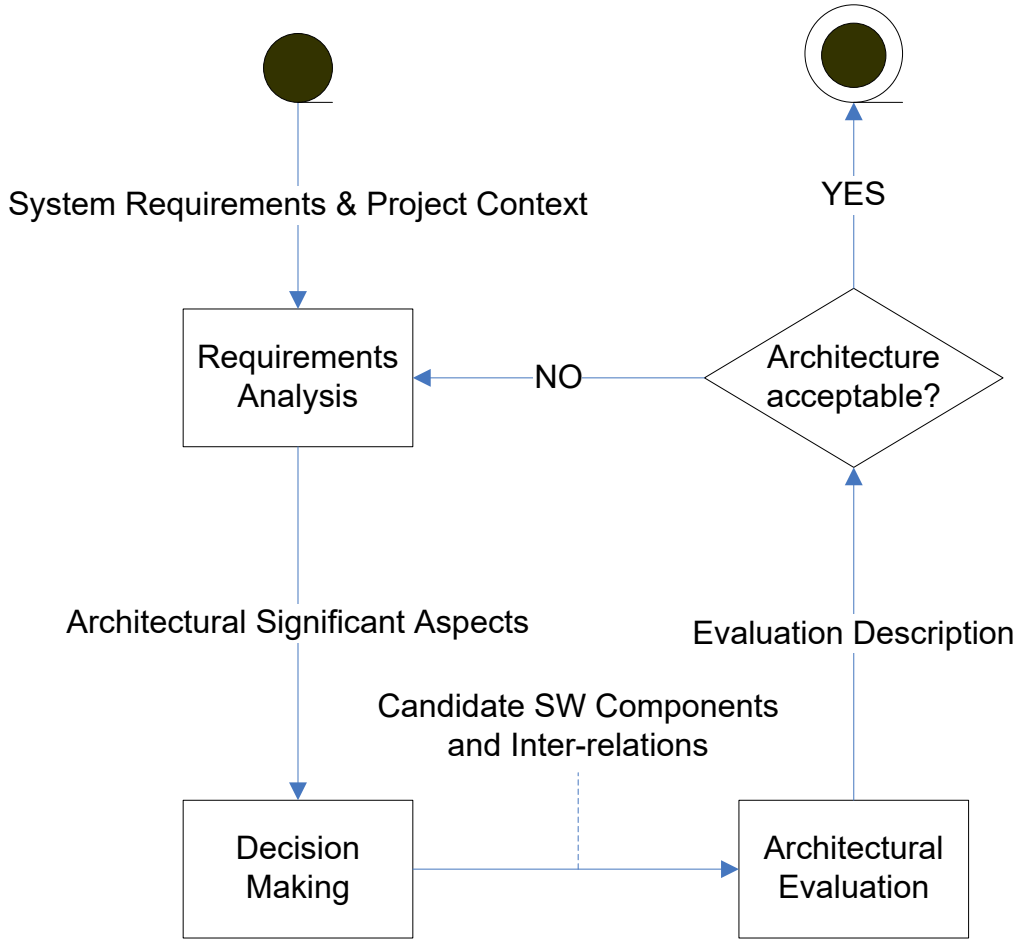
- **Method**: There are a number of languages, process models, and methods that prescribe a systematic technique for bridging the considerable gap between software architecture and requirements.

- **Intuition**: Many architects rely on the invention of software architecture elements and their relations purely based on their own experience.

- The proportion among these three software architecture sources depends on the experience of architects, their educational background, and the novelty of the system.



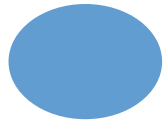
# The General Software Architecture Design Method



# Software architecture design process

## Understand the problem

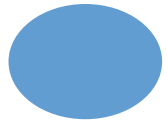
- This phase consists of analyzing the problem and extracting the most critical needs from the big, ambiguous problem description. This phase is largely about *requirements analysis*, focusing on revealing those stakeholders' needs that are architecturally significant.
- This is done by determining the desired *quality attributes* of the system to build, that, together with the *business goals*, drive the architectural decisions.
- The Quality Attribute Workshop is an approach for analyzing and eliciting the requirements that are architecturally significant.



# Software architecture design process

**Find a solution for the problem**

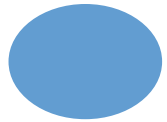
- This phase consists of *decision-making* to fulfill the stakeholders' needs (as defined in the previous phase) by choosing the most appropriate architectural design option(s) from the available alternatives.
- In this phase the properties of software components and their relationships are defined.
- The remaining of these slides are focused on the activities of this phase.



# Software architecture design process

## Evaluate the solution

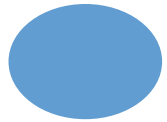
- Finally it is necessary to decide whether and to what degree the chosen alternative solves the problem.
- In the architecture context, this phase consists of *architectural evaluation*.



# Decision-making in software architecture design

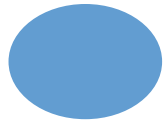
- Because the software architecture design process focuses on deriving an architecture from software requirements, as already mentioned, it includes (but is not merely the application of) a decision-making technique.

- *The decision-making technique focuses on the issue of selecting among a number of alternatives — the tactics — for achieving the system's desired quality attributes.*



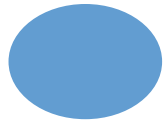
# Software architecture design decisions

- Design decisions are the core of software architecture. The life of a software architect is a long, and sometimes painful, succession of suboptimal decisions made partly in the dark.



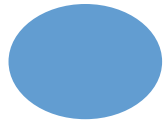
# Software architecture design decisions

- Requirements are frequently captured informally while software architecture is specified formally. Consequently, there is a semantic gap to deal with.
- Quality attribute requirements are difficult to specify in an architectural model.
- Software is often developed in an iterative way. Certain types of requirements cannot be well defined in the early development phases. This implies that sometimes architects have to make their decisions based upon vague requirements.
- Architectural decisions are hard to change after they are made because subsequent decisions are taken based on them; for example, detailed design and programming.



# Software architecture design decisions

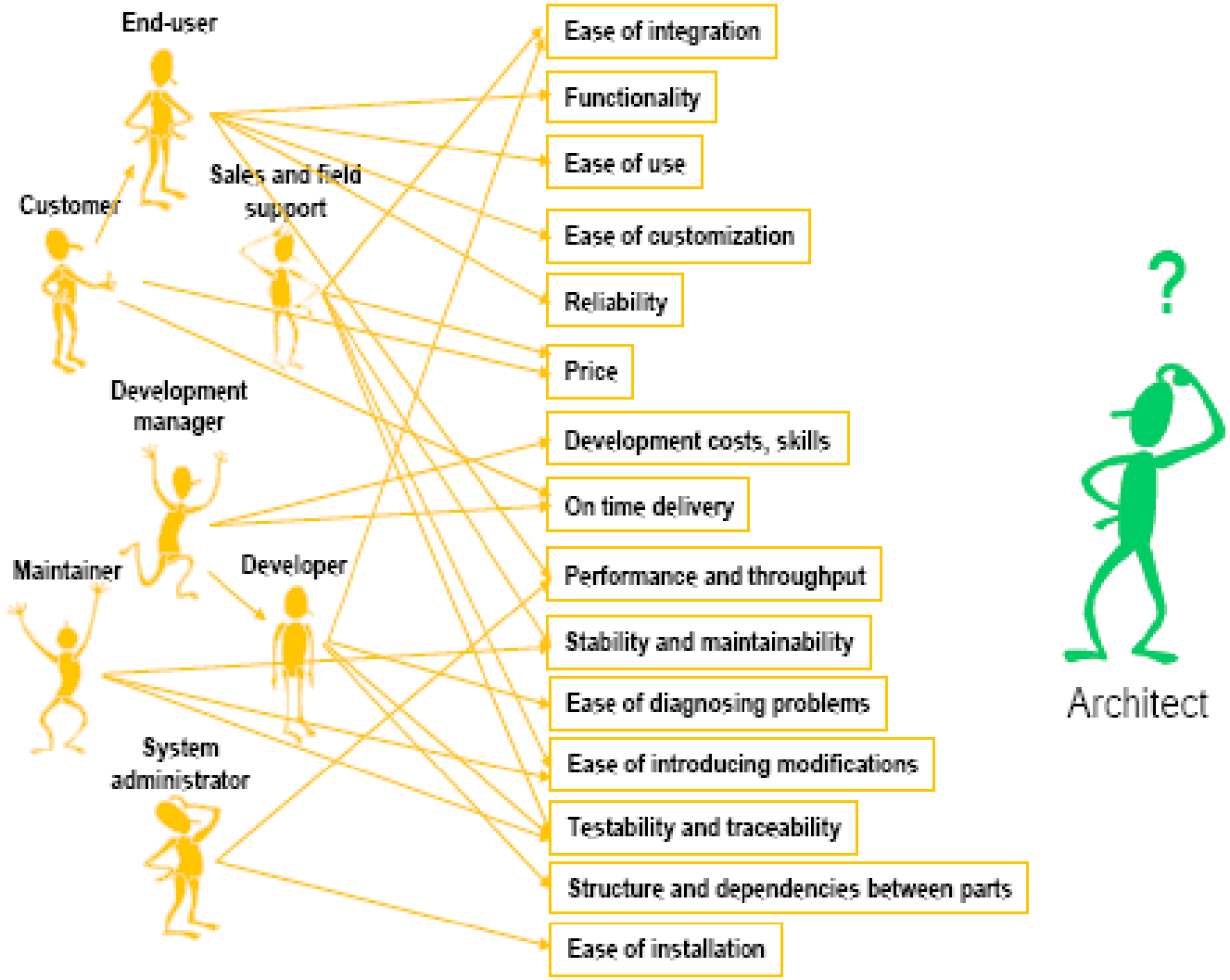
- Stakeholders dealing with software architecture issues view the system from different perspectives; this usually implies conflicting goals and expectations among developers and even differences in terminology. Therefore, architects are requested to find the right tradeoff by balancing the many forces influencing the system development, as well as to work with all the stakeholders.



# Software architecture design decisions



# Software architecture design decisions



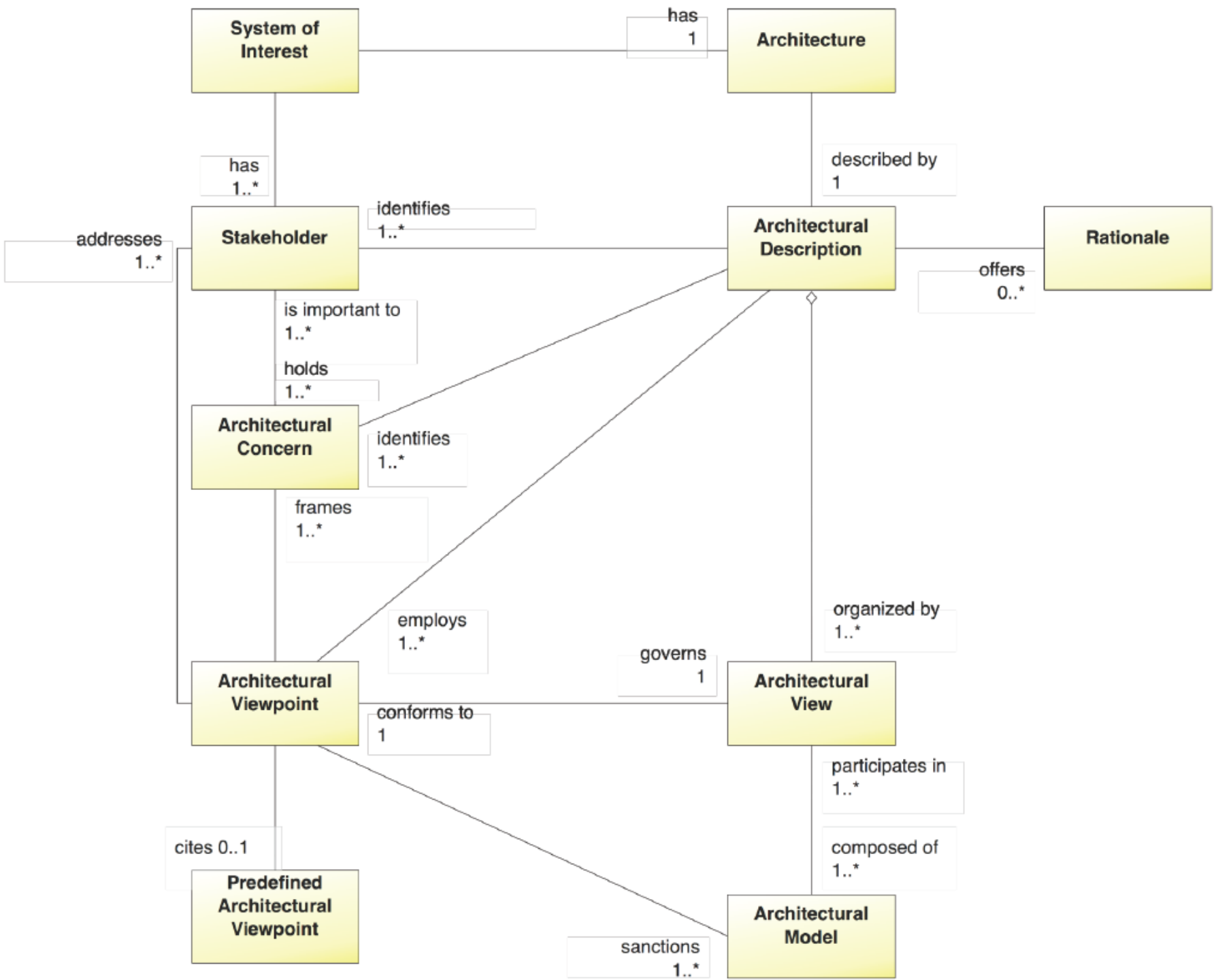
# How to document a SA?

# Definitions

- Architecture description: is organized into one or more architectural views.
- Architectural model: A part of an architectural view. An architectural model may participate in more than one view to enable sharing of concern-related details across views, without repetition.
- Architectural view: A representation of a whole system from the perspective of an identified set of architecture-related concerns. It is composed of one or more architectural models
- Architectural viewpoint: The conventions for the construction, interpretation and use of an architectural view to frame an identified set of architecture-related concerns.

# Relatinos across concepts

- An architectural description shall contain one or more architectural views.
- There shall be exactly one architectural view in an architectural description for each architectural viewpoint used.
- Each architectural view in an architectural description shall be governed by exactly one architectural viewpoint.
- An architectural description may also include other architectural information which is not properly contained in any of its architectural views, as a result of an organization's documentation practices.




Falessi 2009 ©

# Product Lines software engineering

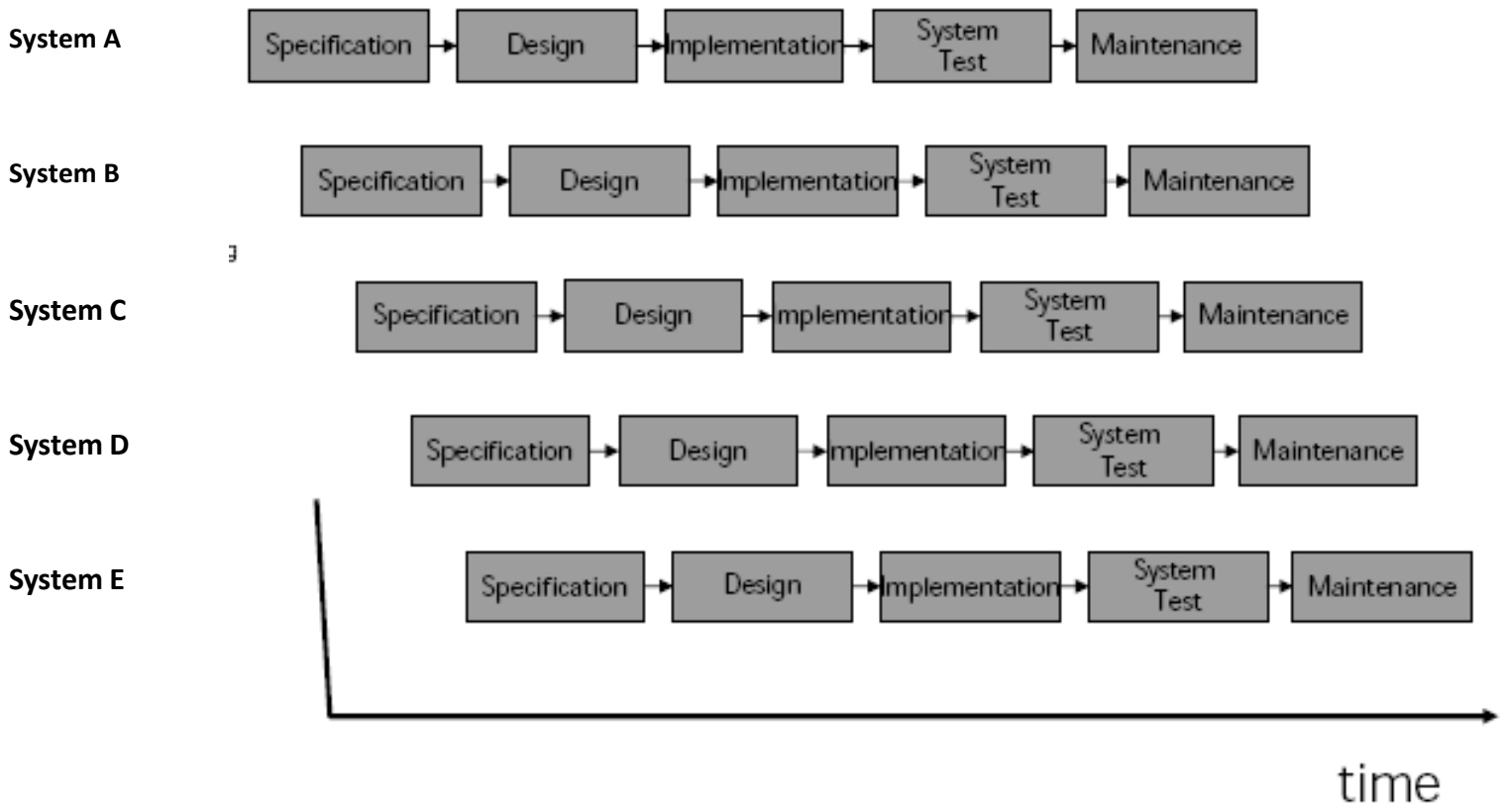
# Reference architecture?

- A Reference Architecture is, in essence, a predefined architectural pattern, or set of patterns, possibly partially or completely instantiated.

# Introduction
## Definition

- "A software product line (SPL) is a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way." Clements & Northrop.

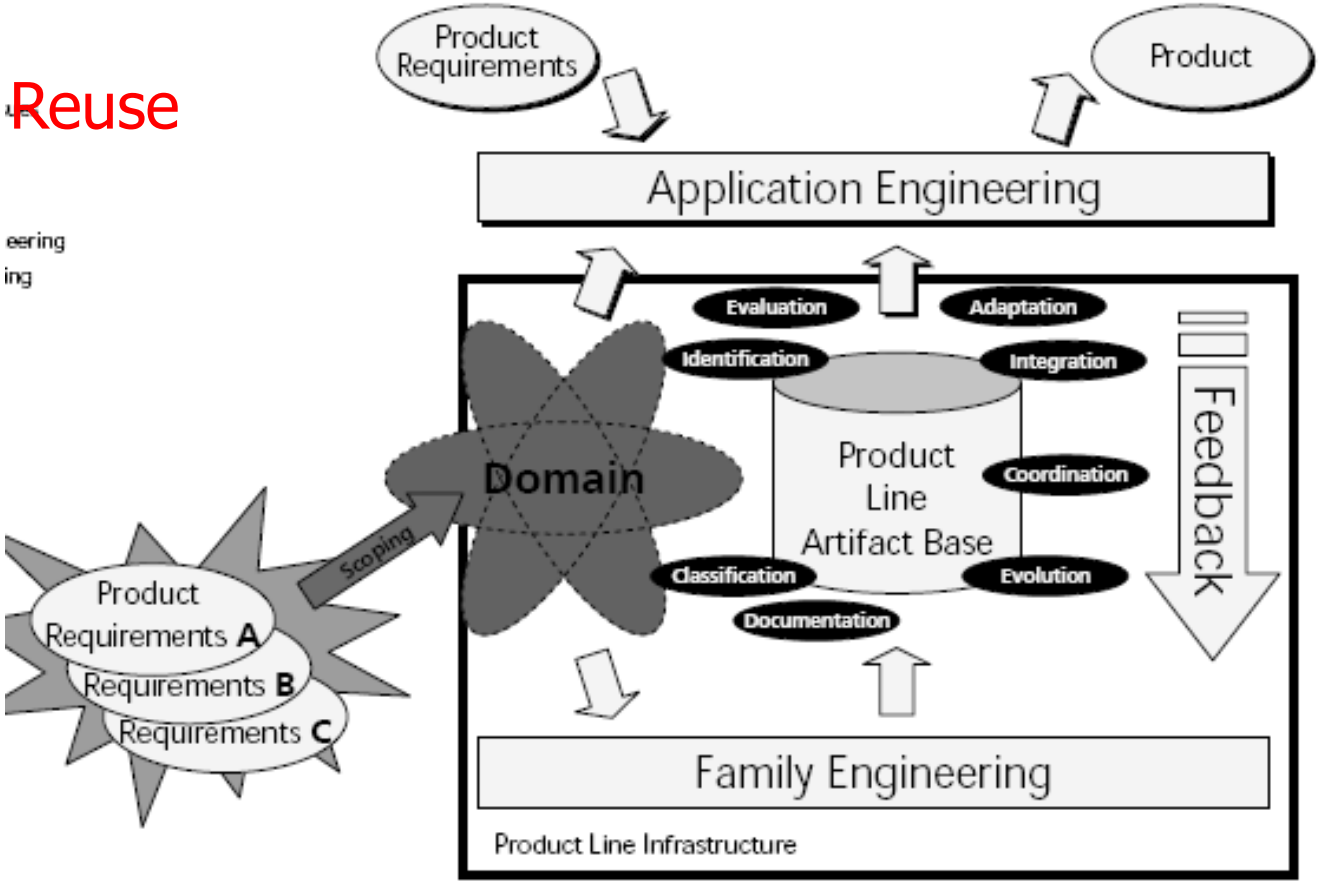
# Individual system development



# Introduction

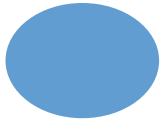
## Product Line Development

Systematic Reuse



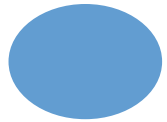
# PL: Introduction

- Product line is a family of products designed to take advantage of their common aspects.
- PL means systematic reuse. i.e. the amount of predicted reuse is the reasons for the necessary up-front investment.
- PL is not only reuse of code! In PL the concept of variability is applied to Requirements, Component models, Tests, Code, etc. This should provide reuse of documentations, test plans, test cases, configuration-control processes, code, performance models, etc. etc.



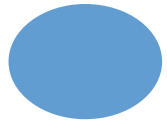
# PL SCOPING

- It focused on defining which functionalities of the products in the PL will be shared among the products.
- A key factor in achieving value from the reuse opportunities is that the investment in reuse is focused on the functionality that promises an optimal return of investment.

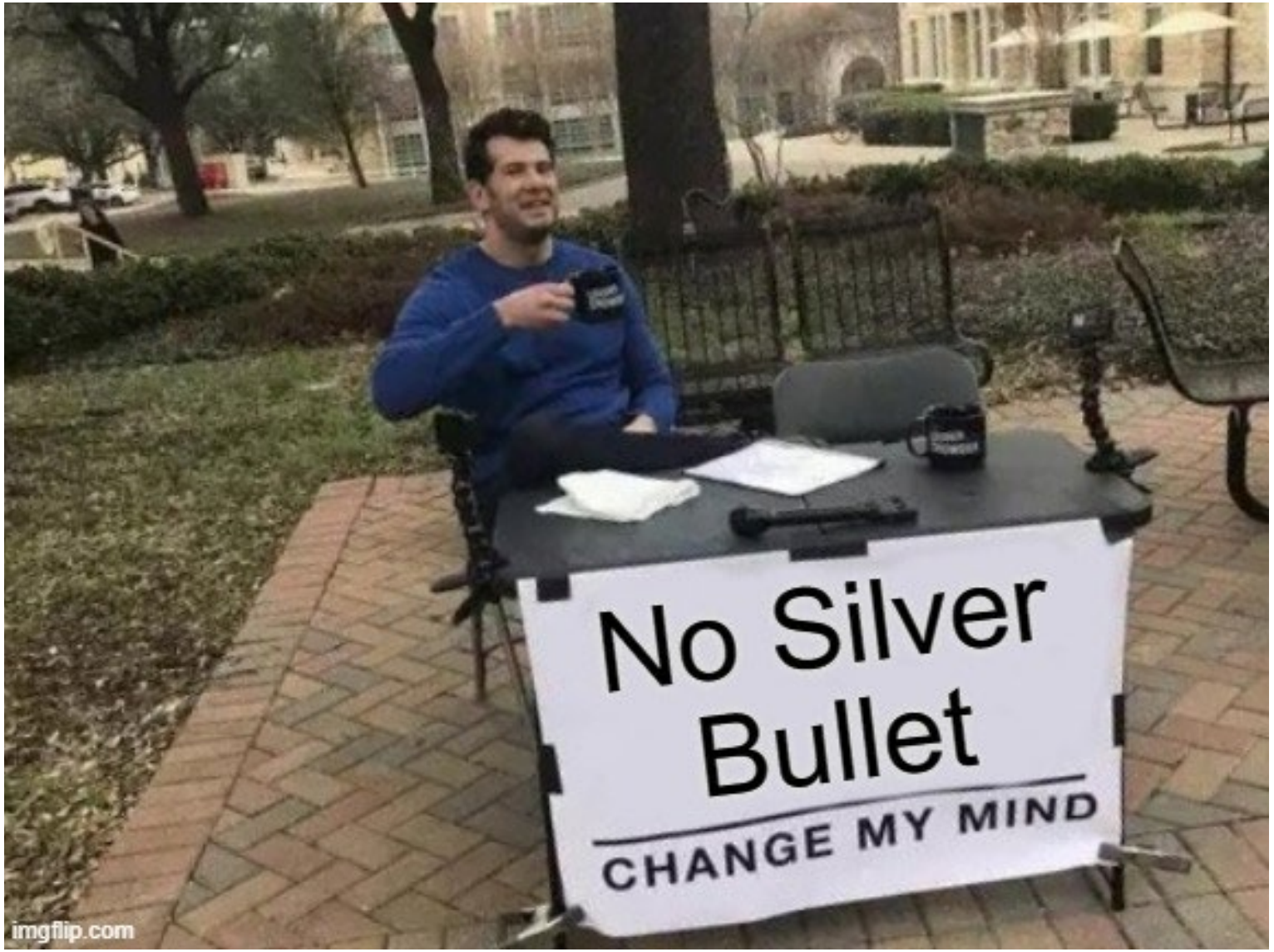


# PL SCOPING

- A too large scope would cause waste of investment on asset that will not be reused.
- A too narrow scope would not allow to take advantage of all the potential reuse (commonalities among products).

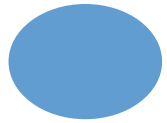


# PL Scope



# Product-line architecture

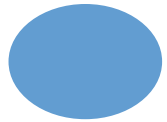
- The product-line architecture (PLA) is the key element of the PL.
- The PLA must **accommodate the commonalities while supporting the variability** in the PL.
- Ideally, individual product features will be provided by specific components.
- The PLA should be one for the entire PL.



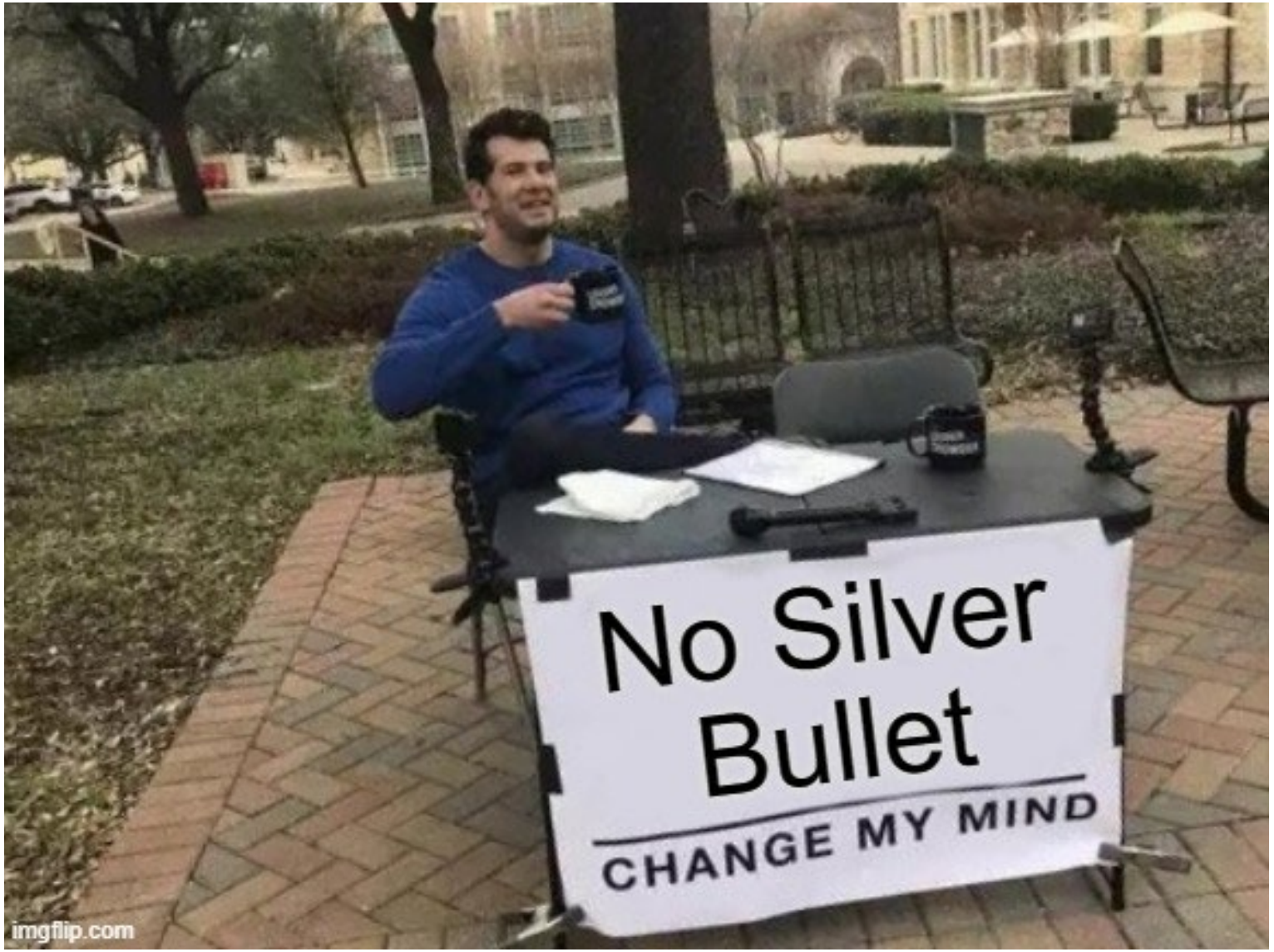
# Potential Drawbacks in developing reusable components

- **Complex interface**: In order to be extended with product specific code, a component must exhibit an interface. Although ideally this interface is clean and reveals minimal details, in practice, in order to support several  extensions, the interface becomes complex.

- **Organizational noise**: a certain level of reuse may require heavy organizational change; this may provoke problems associated with productivity and predictability of schedule, motivation, team spirit, etc. etc.



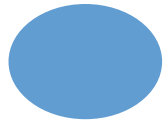
# Reusability



# Architectural Styles

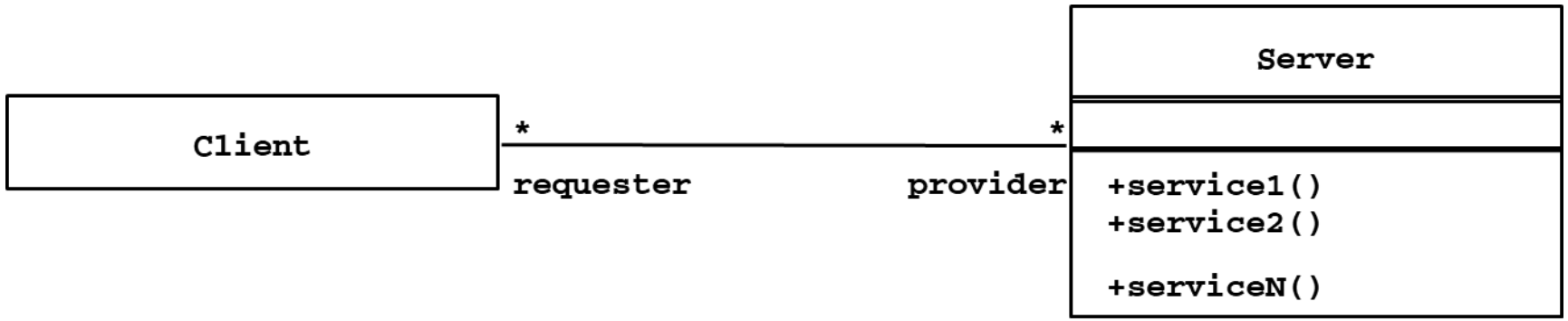
# Architectural Styles

- **Subsystem decomposition**: Identification of subsystems, services, and their relationship to each other.
- **Architectural Style**: A pattern for subsystem decomposition
- **Software Architecture**: Instance of an architectural style



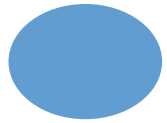
# Client/Server Architectural Style

- One or many servers provide services to instances of subsystems, called clients
- Each client calls on the server, which performs some service and returns the result
  - The clients know the *interface* of the server
  - The server does not need to know the interface of the client
- The response in general is immediate
- End users interact only with the client.



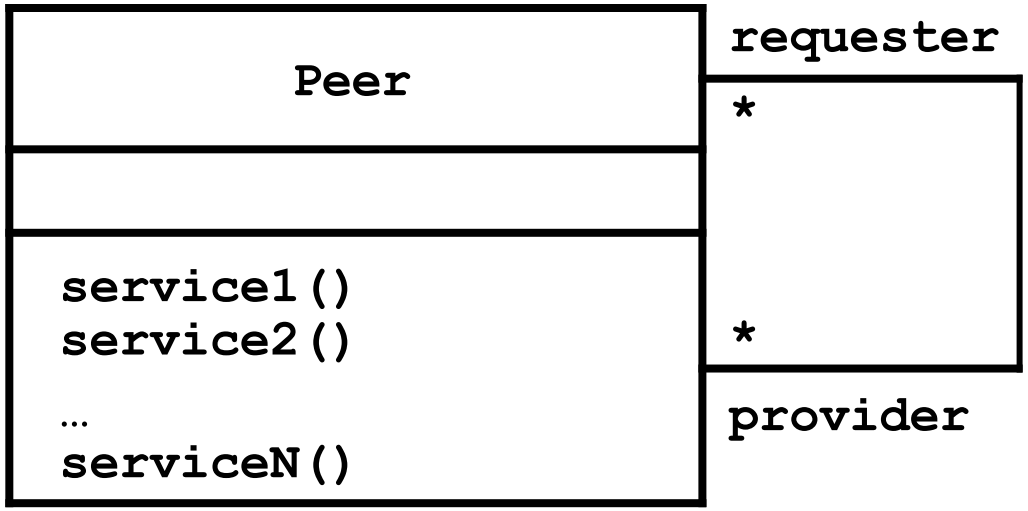
# Client/Server

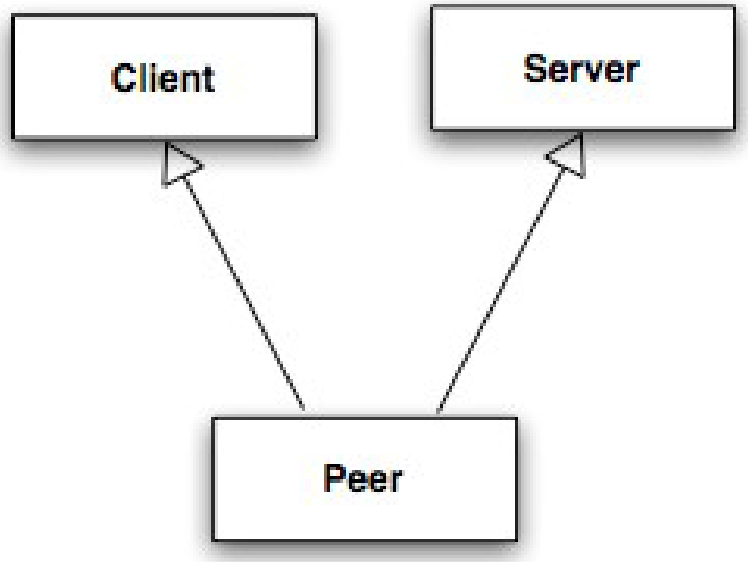
- Often used in the design of database systems
  - Front-end: User application (client)
  - Back end: Database access and manipulation (server)
- Functions performed by client:
  - Input from the user (Customized user interface)
  - Front-end processing of input data
- Functions performed by the database server:
  - Centralized data management
  - Data integrity and database consistency
  - Database security



# Peer to peer

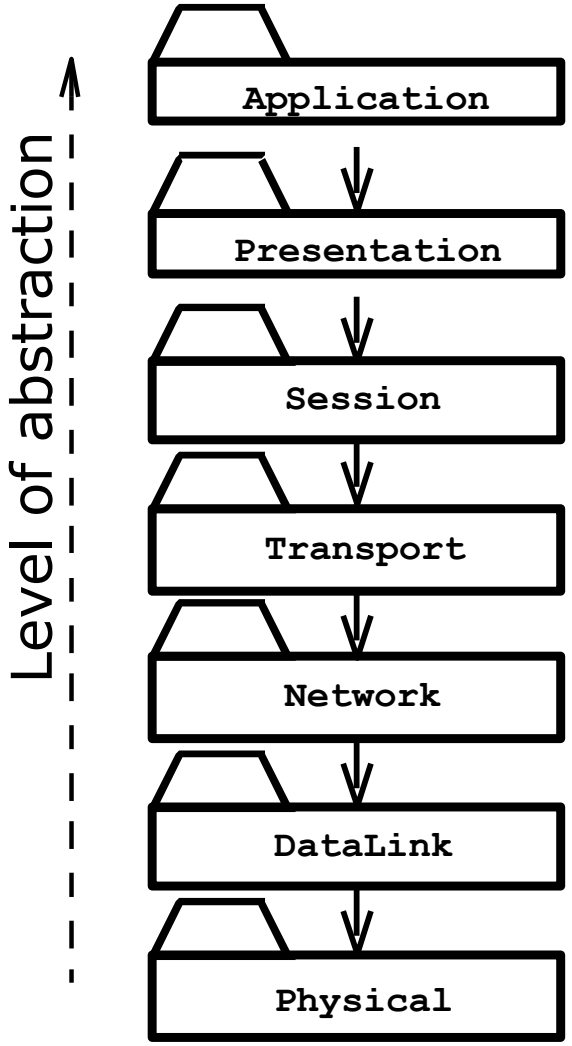
- Generalization of Client/Server Architecture Clients can be servers and servers can be clients
- “A peer can be a client as well as a server”.



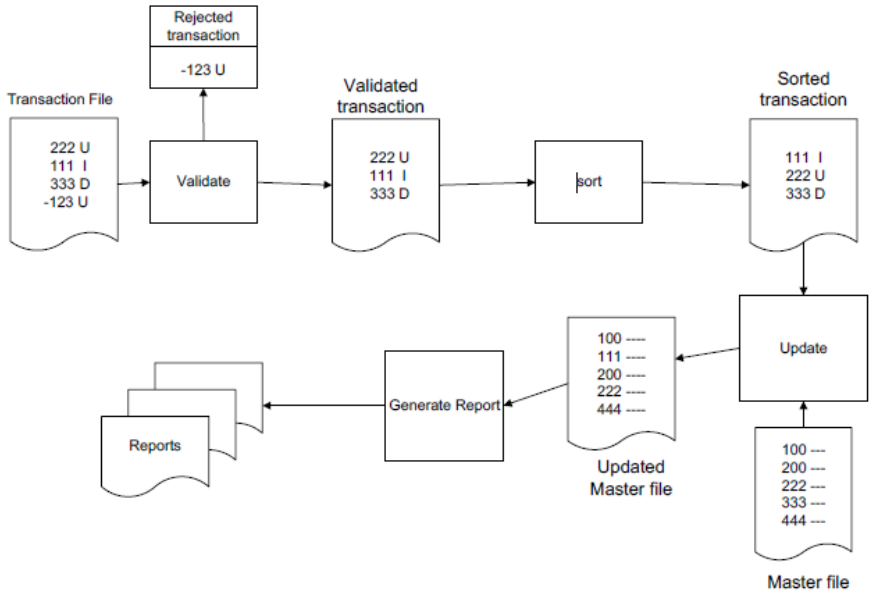
# Example: Peer-to-Peer Architectural Style

- ISO's OSI Reference Model
  - ISO = International Standard Organization
  - OSI = Open System Interconnection
- Reference model which defines 7 layers and communication protocols between the layers



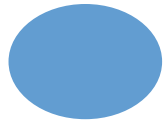
# Pipes and filters

- A component reads streams of data as input and produces streams of data as output.
- Suitable for applications that require a sequence of computations to be performed on data.
- This architectural style focuses on the dynamics (interaction) rather than the structure.



# Shared data

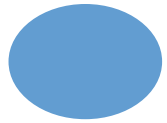
- A number of agents (data consumers/producers) communicate via a centralized data store.
- The primary goal of this style is total decoupling among agents, that can be added or detached at runtime without knowing each other.
- Architectures based on data sharing have the goal of achieving data integration.
- The agents are relatively independent of each other (interact only through the data store).
- The data storage is independent of the agents.



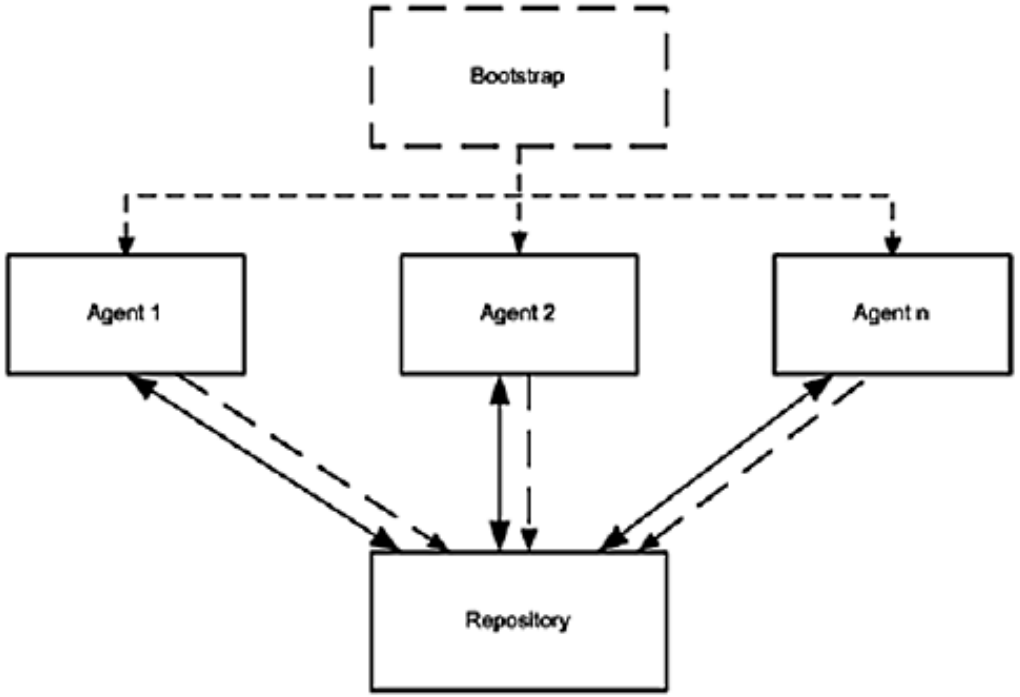
# Shared data

- The control flow within a system based on a shared data store can be managed either by the data store itself or by the agents

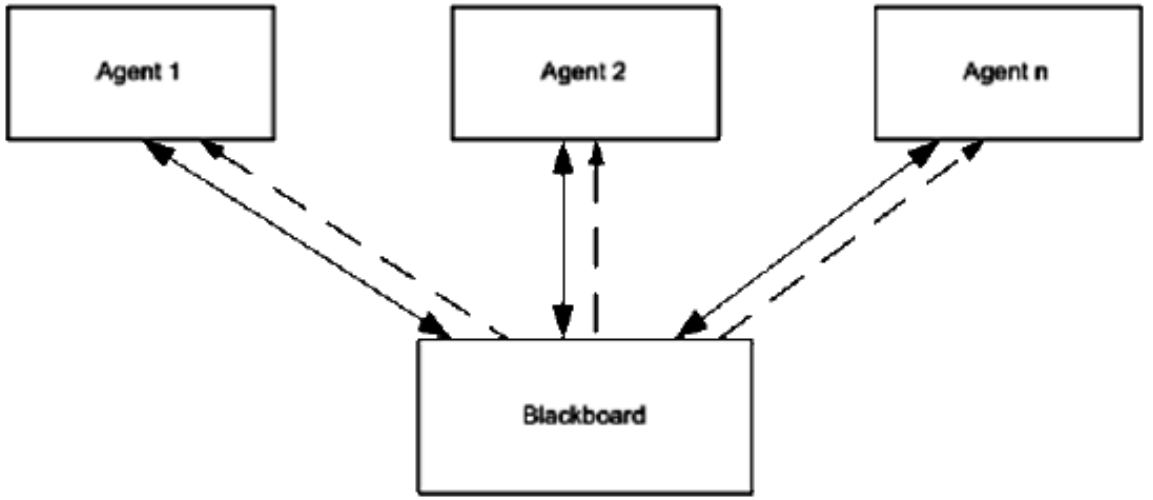
- These two kinds of control flow distinguish the two substyles:
  - Repository (passive): an agent sends a request to the data store to perform some action (e.g. insert data)
  - Blackboard (active): the data store notifies and sends data to subscribing agents when something changes



# Shared data




1. Repository Control Flow




2. Blackboard Control Flow

# References


- Humberto Cervantes and Rick Kazman. 2016. Designing Software Architectures: A Practical Approach (1st. ed.). Addison-Wesley Professional.
- Davide Falessi, Lionel C. Briand, Giovanni Cantone, Rafael Capilla, Philippe Kruchten: The value of design rationale information. ACM Trans. Softw. Eng. Methodol. 22(3): 21:1-21:32 (2013)
- Davide Falessi, Giovanni Cantone, Rick Kazman, Philippe Kruchten: Decision-making techniques for software architecture design: A comparative survey. ACM Comput. Surv. 43(4): 33:1-33:28 (2011)
- Davide Falessi, Giovanni Cantone, Philippe Kruchten: Do Architecture Design Methods Meet Architects' Needs? WICSA 2007: 5